



Lecture 07

From Spaghetti Code to Layers

Basics of Internet Technologies 2

Giving every part of an application one clear job

Learning Objectives

- Recognize when one file or method has too many responsibilities.
- Explain presentation, application, domain, and infrastructure layers.
- Move console input and output into a presentation class.
- Move validation and use cases into an application service.
- Keep the Expense model in the domain layer.
- Keep JSON file access in infrastructure.
- Use Program.cs as a composition root.
- Navigate the new Examples and Project folder structure.

Solution Structure

```
Lecture 07 - From Spaghetti Code to Layers
|-- Layers
|   |-- Lecture07.Application
|   |-- Lecture07.Domain
|   |-- Lecture07.Infrastructure
|   `-- Lecture07.Presentation
|-- Lecture07.CodeExamples
`-- Lecture07.Project
```

- Layers contains the four layer class-library projects.
- Lecture07.CodeExamples is a small runnable experiment.
- Lecture07.Project is the evolving console application that references the layers.

Connection to Lecture 06

- Lecture 06 made the storage boundary explicit with `IExpenseRepository`.
- The project still needs a home for menu code, rules, models, and file code.
- Interfaces help a boundary; layers organize the whole application.
- Today we use the JSON repository from Lecture 06 inside an explicit infrastructure layer.

Three-Hour Lesson Plan

- 20 min - recognize the problem: Program.cs becomes a junk drawer.
- 25 min - responsibilities and four beginner-friendly layers.
- 25 min - domain model and application service.
- 25 min - presentation and infrastructure responsibilities.
- 30 min - composition root and dependency direction.
- 35 min - live refactoring of the expense tracker.
- 20 min - exercises, architecture map, and shared database task.

What Is Spaghetti Code?

- Code is called spaghetti when responsibilities are tangled together.
- One method reads input, validates, calculates, writes JSON, and prints output.
- A small change requires searching through unrelated details.
- The problem is not file length alone; it is unclear ownership.
- We improve structure gradually, not by adding folders for decoration.

Before: One Method With Too Many Jobs

```
void AddExpense()
{
    string name = Console.ReadLine();
    // Validate input
    // Create Expense
    // Add to list
    // Serialize JSON
    // Write file
    // Print message
}
```

- This can work for a first console program.
- As features grow, it becomes hard to test and change.
- Each comment names a different responsibility.

One Change, Many Risks

- Want a database? File code is mixed with menu code.
- Want a web API? Console calls are mixed with application rules.
- Want different validation? Rules are scattered in command handlers.
- Want to test a calculation? Starting the console may be required.
- Layers create places where future changes can live.

A Layer Is a Responsibility Boundary

- A layer groups code with a similar reason to change.
- Presentation changes when the user interface changes.
- Application changes when a use case or business rule changes.
- Infrastructure changes when storage or external technology changes.
- Domain changes when the meaning of the model changes.

Four Layer Projects in Layers

Presentation

- Console menu
- Reads input
- Prints output

Application

- Use cases
- Validation
- Coordinates work

Domain

- Expense model
- Meaningful types
- Core data

Infrastructure

- JSON repository
- File access
- Technology details

Dependency Direction

- Presentation calls Application.
- Application uses Domain and depends on the repository contract.
- Infrastructure implements the repository contract.
- Domain does not know about console menus, JSON, or files.
- Program.cs creates the concrete objects and connects them.

Project Map

```
Program.cs
-> ConsoleMenu (Presentation)
-> ExpenseService (Application)
-> IExpenseRepository (Application contract)
-> JsonExpenseRepository (Infrastructure)
-> expenses.json
```

Expense is used by all layers as a Domain model.

- Arrows mean one part uses the next part.
- The JSON class is chosen only in Program.cs.

Domain: Expense

```
namespace Lecture07.Project.Domain;

public class Expense
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Amount { get; set; }
    public string Category { get; set; }
    public DateTime CreatedAt { get; set; }
}
```

- Expense represents a concept from the problem domain.
- The model uses meaningful types: decimal for money and DateTime for time.
- It does not read files or ask questions in the console.

Why Domain Has No Console Code

- An Expense should mean the same thing in a console app, web API, or test.
- `Console.ReadLine` is a presentation concern.
- `File.WriteAllText` is an infrastructure concern.
- Keeping the model independent makes it reusable.

Application: The Repository Contract

```
public interface IExpenseRepository
{
    List<Expense> GetAll();
    void Add(Expense expense);
    bool DeleteById(int id);
}
```

- The application defines what it needs from storage.
- It names actions in the language of the application.
- It does not mention JSON, file paths, SELECT, or INSERT.

Application: ExpenseService

```
public class ExpenseService
{
    private readonly IExpenseRepository repository;

    public ExpenseService(IExpenseRepository repository)
    {
        this.repository = repository;
    }
}
```

- The service represents application use cases.
- It receives a storage contract in its constructor.
- It coordinates domain objects and repository actions.

Validation Moves Into the Service

```
public bool Add(string name, decimal amount, string category,
    out string message)
{
    if (string.IsNullOrEmpty(name))
    {
        message = "Name cannot be empty.";
        return false;
    }

    if (amount <= 0)
    {
        message = "Amount must be greater than zero.";
        return false;
    }
}
```

- The rule is not tied to a console command.
- A future web API can call the same service method.
- The message is returned so the presentation layer can show it.

Application Service Creates the Model

```
int nextId = expenses.Count == 0
    ? 1
    : expenses.Max(expense => expense.Id) + 1;

repository.Add(new Expense(
    nextId, name, amount, category, DateTime.Now));
```

- The service owns the use case of adding an expense.
- The repository owns storage after Add is called.
- The menu does not decide how an Expense is saved.

Infrastructure: JSON Repository

```
public class JsonExpenseRepository : IExpenseRepository
{
    private readonly string filePath;
    private readonly List<Expense> expenses;

    public JsonExpenseRepository(string filePath)
    {
        this.filePath = filePath;
        expenses = Load();
    }
}
```

- This class owns the JSON and file path details.
- It implements an application contract.
- Later a database repository can replace it.

Infrastructure Owns File Details

```
string json = JsonSerializer.Serialize(expenses, options);  
File.WriteAllText(filePath, json);
```

- System.Text.Json belongs in Infrastructure here.
- Directory creation and IOException handling belong here too.
- The service never needs to know how JSON is written.

Presentation: ConsoleMenu

```
public class ConsoleMenu
{
    private readonly ExpenseService service;

    public ConsoleMenu(ExpenseService service)
    {
        this.service = service;
    }
}
```

- ConsoleMenu owns questions, commands, and printed messages.
- It receives the application service, not a JSON repository.
- The presentation layer should be thin: collect input and display results.

Presentation Converts Input

```
Console.Write("Amount: ");
string amountText = Console.ReadLine();

if (!decimal.TryParse(amountText, out decimal amount))
{
    Console.WriteLine("Amount must be a number.");
    return;
}

service.Add(name, amount, category, out string message);
```

- Parsing text is needed because the console gives us strings.
- The menu handles parsing feedback.
- The service handles whether the parsed data is valid for the use case.

Thin Presentation, Useful Application

ConsoleMenu

- Read input
- Parse numbers
- Call service
- Print result

ExpenseService

- Validate rules
- Create Expense
- Choose ids
- Request storage

Program.cs Is the Composition Root

```
interface IRepository {  
    void AddExpense(Expense expense);  
    Expense[] GetExpenses();  
}  
  
IExpenseRepository repository =  
    new JsonExpenseRepository(dataPath);  
ExpenseService service = new ExpenseService(repository);  
ConsoleMenu menu = new ConsoleMenu(service);  
  
menu.Run();
```

- Composition root means the place that creates and connects concrete objects.
- Only this file chooses `JsonExpenseRepository`.
- This is simple manual dependency injection; a framework comes much later.

Why Separate Layer Projects?

- Each project makes a dependency visible and enforceable at compile time.
- Domain has no reference to console or JSON technology.
- Application references Domain; Infrastructure and Presentation reference only what they need.
- Lecture07.Project is the composition point that references the concrete layers.

Three Kinds of Projects

Layers

- Four class libraries
- One responsibility each
- Reusable application parts
- Dependencies are explicit

CodeExamples

- Small runnable snippets
- One concept at a time
- Safe place to experiment
- No application architecture required

Project

- Console application
- References the layers
- Composition root in Program.cs
- Preserves user features

Refactoring Sequence

- 1. Keep the working console project as a reference.
- 2. Create the Domain/Expense class file.
- 3. Move IExpenseRepository and ExpenseService into Application.
- 4. Move JSON code into Infrastructure/JsonExpenseRepository.
- 5. Move the menu loop into Presentation/ConsoleMenu.
- 6. Leave Program.cs only to connect objects and run the menu.
- 7. Build and test after each move.

Before and After

Before

- Program.cs knows everything
- JSON and menu intertwined
- Rules repeat in commands
- Future changes spread widely

After

- Each class has one main job
- JSON isolated in Infrastructure
- Rules are in ExpenseService
- Future UI or storage changes are local

Common Mistakes

- Putting `Console.WriteLine` inside `ExpenseService` or `Expense`.
- Letting `ConsoleMenu` call `File.WriteAllText` directly.
- Putting all code in folders but keeping the same tangled dependencies.
- Creating a service with no real use case logic.
- Making Domain depend on JSON or the console.
- Splitting into dozens of folders before there is a need.

Guided Exercise A

- Create a TaskItem domain class with Id, Title, IsCompleted, and CreatedAt.
- Create ITaskRepository with GetAll, Add, and DeleteById.
- Create TaskService with validation for a non-empty title.
- Create a small ConsoleMenu method that calls the service.
- Describe which file belongs in each layer.

Guided Exercise B

- Take one method from a previous project that does three or more jobs.
- List its responsibilities in comments.
- Move one storage responsibility behind an interface and repository.
- Move one validation rule into a service method.
- Build after every move and explain what became easier to change.

Student Project Increment

- Create a Layers solution folder with four class-library projects.
- Put the entity in LectureNN.Domain and the service plus contracts in LectureNN.Application.
- Put JSON or SQL technology in LectureNN.Infrastructure.
- Put console interaction in LectureNN.Presentation.
- Create a separate LectureNN.Project console application that references the layers.
- Keep its Program.cs as the composition root.

Shared Database-Course Increment

- Use the same table designed in the database course.
- Map future SQL code to Infrastructure, not to ConsoleMenu or ExpenseService.
- Keep IExpenseRepository in Application because it describes what the app needs.
- Write a short diagram showing Console -> Service -> Repository -> Database.
- The actual SQL connection begins in a later lecture.

Exit Ticket

- Which layer should own `Console.ReadLine`?
- Which layer should own JSON serialization?
- Why does `ExpenseService` depend on `IExpenseRepository` instead of JSON?
- What is the one job of `Program.cs` now?
- What will change first if we replace the console with a web API?



Next Time

Services, Repositories, Validation and Multiple Users

We will deepen the Application layer with real use cases and ownership.
The layered console becomes Milestone 03.